

Lab 1: Sorting, Complexity

In this lab, you will explore how different sorting algorithms perform in practice. There are two main tasks:

- Measuring the runtime of various sorting algorithms on different degrees of sortedness of input data and empirically determining their complexity
- Taking a simple implementation of quicksort and speeding it up by adding important optimisations

About the labs

- The lab is part of the examination of the course. Therefore, you must not copy code from or show code to other groups. You are welcome to discuss ideas with one another, but anything you do must be **the work of you and your lab partners**.

Getting started

Make sure you have access to a Java compiler or IDE. If you are using the MUST lab machines, you should (hopefully) have access to the command-line compiler `javac`, as well as the Visual Studio Code, Netbeans, IntelliJ and Eclipse IDEs - use whichever you prefer.

The lab files contain implementations of three sorting algorithms:

- **Insertion.java** - insertion sort
- **Merge.java** - top-down merge sort
- **Quick.java** - quicksort, using the first element as the pivot

as well as one more useful class:

- **Bench.java** - a benchmark and testing program;
prints performance reports for the sorting algorithms
and tests that they work correctly.

and a file **answers.txt** where you will write down answers to questions in the lab.

Now compile and run **Bench.java**. It will measure the performance of each of the algorithms above and print out a timing report for each one. The timing report will look something like this:

Quick.java: quicksort (times in microseconds)

Size	Random		95% sorted		Sorted	
	Total	per item	Total	per item	Total	per item
10	0.1	0.0068	0.1	0.0077	0.1	0.0083
30	0.3	0.0086	0.4	0.0129	0.4	0.0132
100	1.3	0.0126	2.1	0.0212	2.5	0.0253
300	4.7	0.0156	7.5	0.0251	17.6	0.0587
1000	32.6	0.0326	28.0	0.0280	162.9	0.1629
3000	202.4	0.0675	225.0	0.0750	1439.5	0.4798
10000	752.1	0.0752	789.7	0.0790	13869.3	1.3869
30000	2454.5	0.0818	3498.0	0.1166	129850.6	4.3284
100000	9098.2	0.0910	10966.2	0.1097	1467753.1	14.6775

At the top of the table you see which sorting algorithm was tested. The table shows how the algorithm's runtime (given in microseconds) varies with the size of the input array (shown in the leftmost column) for three degrees of sortedness:

- Completely random arrays
- Arrays where 95% of elements are in the right order, but the rest are chosen at random
- Arrays that are already in the right order

The time is presented in two columns:

- *Total*: This shows the total time for sorting that array.
- *Per item*: This is the total time divided by the size of the array.

For example, the table above shows that quicksort took 752μs on an array of 100000 random integers, and 1.47s on a sorted array of 100000 integers.

Task 1: figuring out the complexity

Look at the timing reports for the different algorithms.

You will notice that some of the algorithms are more efficient than others. In fact, the algorithms have different *complexities*:

- Some take quadratic time (the runtime is proportional to n^2 , where n is the size of the input array).
- Others take linearithmic time (the runtime is proportional to $n \log n$).
- Others take linear time (the runtime is proportional to n).

- Often the complexity even depends on the degree of sortedness of the input array (random, 95% sorted, or sorted).

We will learn more about complexity later in the course.

For now, we take the above as defining complexity.

Since complexity ignores a factor of proportionality, it is not a direct measure of speed.

Rather, it tracks changes in a program's runtime as its input becomes larger.

Your first task is to answer the following question:

- For each of the three sorting algorithms, and each kind of array (random, 95% sorted, sorted), what is the algorithm's complexity?

You should answer this question *only looking at the timing data*, not thinking about how the algorithms work.

Hint: The "time per item" column is useful for finding the complexity. For an array of size n , this column gives the runtime divided by n . So:

- If the runtime is proportional to n^2 ,
the time per item will be proportional to n .
- If the runtime is proportional to $n \log n$,
the time per item will be proportional to $\log n$.
- If the runtime is proportional to n ,
the time per item will be roughly a constant regardless of the array size.

Write your answers in the file **answers.txt** provided. One answer is filled in for you already.

Task 2: improving quicksort

You will have noticed that the quicksort implementation in **Quick.java** performs badly on sorted input arrays. This is because it always chooses the pivot to be the first element of the input array, which in a sorted array means that one of the partitions is always empty. In this task you will make several improvements to the quicksort implementation and see how well they work in practice.

If you look in **Quick.java**, you will find three comments starting with `// TODO: [...]` which indicate an improvement you should make. **Before running your modified code, read the next section, "Choosing which improvements get used"!** Here are the three improvements:

1. **Before sorting the array, shuffle it** (rearrange it into a random order). You should only do this immediately at the beginning of quicksort, and not inside any of the recursive calls. To shuffle the

input array, you can call the `void shuffle(int[] array)` method which is already defined in `Quick.java`.

The goal of shuffling the input is to avoid having bad performance on sorted arrays, by turning every input array into a randomly-ordered array. With the array shuffled, it should be OK to always use the first element as the pivot.

2. **Use the median-of-three for the pivot selection.** By default, the first element is used as pivot in the subarray we are currently sorting. Instead, look at the first element, the last element and the middle element of the subarray, and use the median value of those three as the pivot.

In **Quick.java**, you will find a method:

```
int medianOfThree(int[] a, int i, int j, int k)
```

which looks at `a[i]`, `a[j]` and `a[k]` and return the index (`i`, `j` or `k`) containing the median element.

Change the partitioning method so that it first finds the median-of-three and then *swaps* it with the first element in the subarray. To do the swap, use the method

```
void exchange(int[] a, int i, int j)
```

which is defined in `Quick.java`.

The goal of median-of-three is twofold: 1) ensure that if the subarray is sorted, the middle value is chosen as the pivot; 2) by choosing between three pivot candidates, try to improve the distribution of elements between the two partitions.

3. **Switch to insertion sort for small subarrays.** Quicksort recurses all the way down until it reaches a subarray of size 1, which results in very many recursive calls. You should change the recursion's base case so that small subarrays are sorted with insertion sort. (What in the timing data suggests that this is a good idea?)

To sort the subarray `a[lo..hi]`, you can call `Insertion.sort(a, lo, hi)`. **You do not need to implement insertion sort yourself.**

You will need to choose an appropriate cutoff size for switching to insertion sort. You should **do this in a systematic way**, rather than guessing. For example, here is one approach you could take:

- From the timing data, figure out an initial estimate for the cutoff value.
- To refine your estimate, try some nearby values and measure their performance.

But you may find a good cutoff value in whatever way you like. Also, you do not need to find an exact value, as this will differ from computer to computer - rounding the cutoff value to the nearest 10 would be reasonable.

Whatever approach you use, make sure that your answer is reasonable, by checking that when you change the cutoff value by some amount in either direction, the performance starts to get worse.

Choosing which improvements get used

When you benchmark quicksort, you will want to sometimes switch different improvements on or off. The **Quick** class therefore allows its caller to customise what improvements get used.

The way this works is that **Quick** has a constructor which takes parameters describing the improvements to be used:

```
public Quick(boolean shuffleFirst, boolean useMedianOfThree, int insertionSortCutoff) { ... }
```

Here:

- If `shuffleFirst` is true, the array is shuffled before sorting.
- If `useMedianOfThree` is true, the median-of-three strategy is used for pivot selection.
- The `insertionSortCutoff` field determines the size at which recursive subcalls in quicksort switches to insertion sort.

The code you were given already takes some of these variables into account.

For example, looking at the code structure, the array should only be shuffled if the `shuffleFirst` variable is true.

This means that it is not enough to implement the three improvements -

you also need to make sure they are enabled when quicksort is called!

The place to do this is in `Bench.java`. In the `main` method at the top of the file, you will see the line

```
executionTimeReport("Quick.java: quicksort", new Quick(false, false, 0)::sort);
```

Note that it says `new Quick(false, false, 0)`. This specifies that the array is not shuffled (the first parameter), median-of-three is not used (the second parameter), and the cutoff to insertion sort is a subarray of size 0 (which effectively disables the cutoff).

The line below (currently commented out) runs quicksort with different parameters:

```
executionTimeReport("Quick.java: quicksort with all improvements", new Quick(true, true, 42)::s
```

This line specifies that the array should be shuffled, that median-of-three should be used for the pivot, and that a cutoff to insertion sort of 42 should be used.

When benchmarking your code, you will want to try out different combinations of improvements, and different cutoffs to insertion sort. To do so, just have several calls to `executionTimeReport`. You can copy the line above, changing the arguments to `Quick` to adjust the settings, and changing the string so that you can tell each version apart.

To help the person grading your lab, please also keep the line that benchmarks quicksort with no improvements (`Quick(false, false, 0)`).

Finding bugs

Bench.java not only measures the performance of your code, it also checks that it sorts correctly. If you introduce a bug when improving quicksort, you will get an error message that looks something like this:

```
Test failed! There is a bug in the sorting algorithm.
Input array: {2, 4}
Expected answer: {2, 4}
Actual answer: {4, 2}
```

This shows that we tried to sort the array `{2, 4}`, which should give the result `{2, 4}`. However, quicksort mistakenly gave the answer `{4, 2}`.

Once you have this test case, you can add a `main` method to **Quick.java** that just sorts the array `{4, 2}`. Then you can for example step through it in an IDE to see what goes wrong.

Your task

Your task is as follows:

- Make the improvements above, one at a time. After you have made an improvement, test it with **Bench.java** and check: did the performance improve? Did the complexity change?

- Figure out which *combination* of improvements works best. That is, it may not be a good idea to add all three! Try different combinations to find the best one.

When you have done that, fill in **answers.txt** with the following information:

- Which of the three improvements above affect the complexity of quicksort, and in what way?
- What cutoff works best for insertion sort? You will also need to explain, in a few sentences, how you found the cutoff value.
- What did you find to be the best combination of improvements?

Your submission

Submit the following files with your changes in a zipped folder with the name of your group e.g. alpha.zip

- Your improved version of **Quick.java**
- A file **output.txt** with the output from running Bench.java on the three different sorting algorithms with the improvements you have chosen for quicksort
- The file **answers.txt**, with all answers filled in

Optional tasks

If you would like an extra challenge, here are some suggestions for things you could do:

- Modify **Quick.java** to choose a random element as pivot. You may think that this would have the same effect as shuffling the array. It doesn't! Try it out and see what happens, then figure out what causes the difference. (Hint: the time taken by partitioning depends on the order of the input array.)
- The merge sort implementation **Merge.java** is also unoptimised. Optimise it! Page 275 of the course book describes an optimisation that makes merge sort take linear time on sorted input ("Test whether the array is already in order"), which you could try out. You can also switch to insertion sort for smaller subarrays, like in quicksort.
- Alternatively, implement a different sorting algorithm! Here are some suggestions, in rough order of difficulty:
 - [Dual-pivot quicksort](#), which is the main algorithm used by `Arrays.sort` when sorting arrays of ints
 - [American flag sort](#), which is a sorting algorithm specialised to sorting arrays of integers. It has the reputation of being very fast. It is a variation of the algorithm "MSD string sort" from section 5.1 of the book, so you may want to read up on that first
 - Run-based mergesort

- [Timsort](#), a very fast sorting algorithm. It is basically run-based mergesort plus lots and lots of clever tricks to make it faster. This is the algorithm used by `Arrays.sort` when sorting arrays of objects, and is also the sorting algorithm used in Python.
- If you are into functional programming, implement some sorting algorithms in a functional language. Run-based mergesort is used by e.g. Haskell and Erlang.
- Make a sorting function that dynamically chooses between different algorithms depending on the input array. `Arrays.sort()` does this: it usually uses dual-pivot quicksort, but switches to a run-based mergesort for input arrays having few runs (and does this check in the recursive calls too).